

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..29

WEB SERVER MIDDLEWARE

 Copy as Markdown

Web server middleware

Routes handle the core logic of your application, but many concerns cut across multiple routes: logging every request, validating API keys, handling CORS headers, or catching errors. Rather than duplicating this code in each route, middleware lets you apply it globally or to specific path prefixes.

Middleware functions are wrappers that sit between the incoming request and your route handler. They can inspect or modify requests before they reach your routes, and transform responses before they're sent back to the client.

The middleware on this page applies only to web server routes. For middleware around your endpoint methods, see [Endpoint middleware](#).

Adding middleware

Use the `addMiddleware` method to apply middleware to specific path prefixes:

```
// Apply to all routes below `/path`
pod.webServer.addMiddleware(myMiddleware, '/path');
```

Host-specific middleware

You can restrict middleware to a specific virtual host by providing the optional `host` parameter:

```
// Middleware for a specific host
pod.webServer.addMiddleware(
  apiKeyMiddleware,
  '/api',
  host: 'api.example.com',
);

// Middleware for all hosts (default behavior)
```



```
pod.webServer.addMiddleware(loggingMiddleware, '/');
```

When `host` is `null` (the default), the middleware applies to all hosts. When set, the middleware only applies to requests with a matching `Host` header.

Creating custom middleware

Middleware is a function that takes a `Handler` and returns a new `Handler`. Here's a simple example that validates API keys for protected routes:

```
Handler apiKeyMiddleware(Handler next) {
  return (Request request) async {
    // Check for API key in header
    final apiKey = request.headers['X-API-Key']?.firstOrNull;

    if (apiKey == null) {
      return Response.unauthorized(
        body: Body.fromString('API key required'),
      );
    }

    // Verify API key
    if (!await isValidApiKey(apiKey)) {
      return Response.forbidden(
        body: Body.fromString('Invalid API key'),
      );
    }

    // Continue to the next handler
    return await next(request);
  };
}

// Apply to protected routes
pod.webServer.addMiddleware(apiKeyMiddleware, '/api');
```

INFO

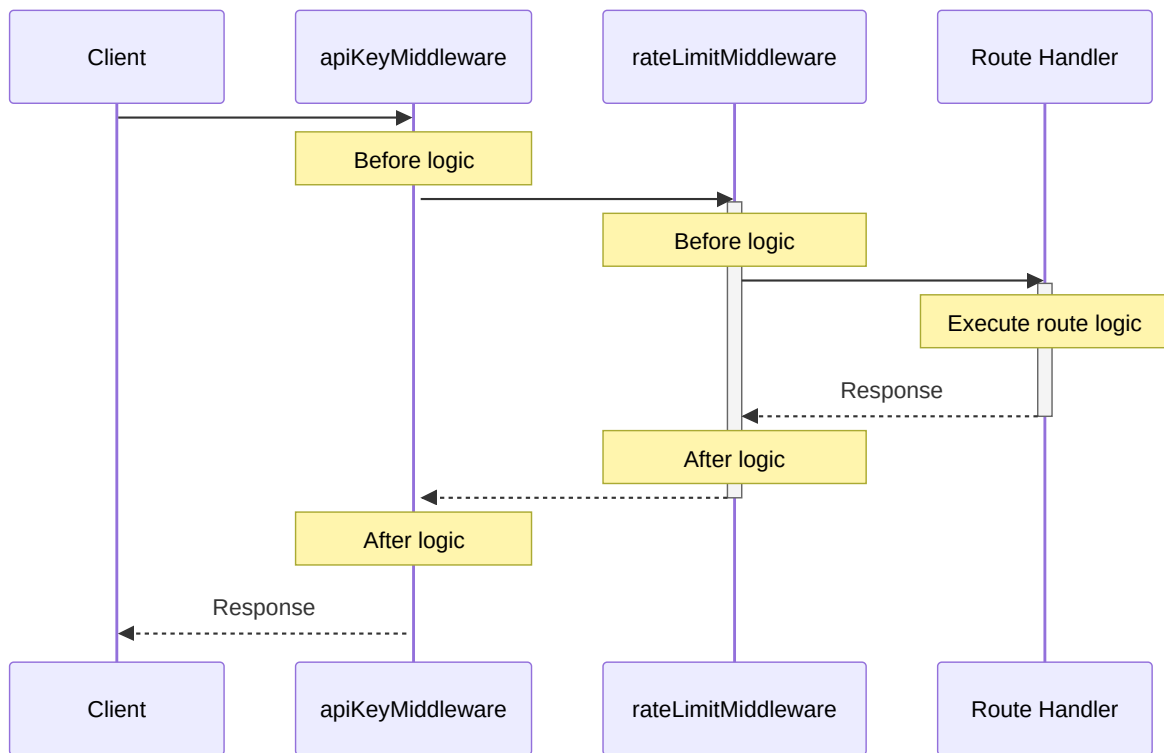
For user authentication, use Serverpod's built-in [authentication](#) system, which integrates with the `Session` object your route's `handleCall` receives. The middleware examples here are for additional web-specific validations like API keys, rate limiting, or request validation.

Middleware execution order

Middleware wraps your routes in layers based on path hierarchy. Middleware registered on a broader path runs first, and middleware on a more specific path runs closer to the route handler. Within the same path, middleware executes in the order it was registered:

```
pod.webServer.addMiddleware(rateLimitMiddleware, '/api/users'); // Runs second for /api/users/list
pod.webServer.addMiddleware(apiKeyMiddleware, '/api');           // Runs first for /api/users/list
```

For a request to `/api/users/list`, the execution order is:



Request-scoped data

Middleware often needs to pass computed data to downstream handlers. For example, a tenant identification middleware might extract the tenant ID from a subdomain, or a logging middleware might generate a request ID for tracing. Since `Request` objects are immutable, you can't just add properties to them. This is where `ContextProperty` comes in.

A `ContextProperty<T>` provides a type-safe way to attach data to a `Request` object without modifying it. Think of it as a side channel for request-scoped data that middleware can write to and routes can read from. The data is automatically cleaned up when the request completes. For more details, see the [Relic documentation](#).

INFO

Serverpod's `Route.handleCall()` already receives a `Session` parameter which includes authenticated user information if available. Use `ContextProperty` for web-specific request data that

isn't part of the standard Session, such as request IDs, feature flags, or API version information extracted from headers.

Creating a `ContextProperty`

Define a `ContextProperty` as a top-level static field:

```
// Define a private context property.
final _tenantProperty = ContextProperty<String>('tenant');

// Create a public getter extension to allow handlers and other middleware to
// read, but not modify the context property.
extension TenantRequestEx on Request {
    String get tenant => _tenantProperty.get(this); // get() throws when unset; [] returns
    null instead.
}
```

Setting values in middleware

Middleware can set values on the context property, making them available to all downstream handlers:

```
// Tenant identification middleware (extracts from subdomain)
Handler tenantMiddleware(Handler next) {
    return (Request request) async {
        final host = request.headers.host;

        // Validate tenant exists (implement your own logic)
        final session = await request.session;
        final tenant = await extractAndValidateTenant(session, host);

        if (tenant == null) {
            return Response.notFound(
                body: Body.fromString('Tenant not found'),
            );
        }

        // Attach tenant to context
        _tenantProperty[request] = tenant;

        return await next(request);
    };
}
```

Accessing values in routes

Route handlers can retrieve the value from the context property:

```
// Routes automatically have access to the tenant
class TenantDataRoute extends Route {
    @override
```

```
Future<Result> handleCall(Session session, Request request) async {
  final tenant = request.tenant; // using the previously defined extension

  // Fetch tenant-specific data
  final data = await Product.db.find(
    session,
    where: (t) => t.tenantId.equals(tenant),
  );

  return Response.ok(
    body: Body.fromString(
      jsonEncode(data.map((p) => p.toJson()).toList()),
      mimeType: MimeType.json,
    ),
  );
}
```

Next steps

- **Request data** - Access path parameters, query parameters, headers, and body
- **Static files** - Serve static assets
- **Server-side HTML** - Render HTML on the server